

LeetCode 1278: Palindrome Partitioning III

Authored by: Arpit Raj, LNMIIT Jaipur

Date: August 26, 2026

Concept & Logic Breakdown

📌 basically to dp partition method hi use krunga isme. `dp[i][k]` waala

📌 but isme ek additional cheez he which is to find `cost to make palindrome`

📌 basically `cost[l][r] = cost[l+1][r-1]`

📌 if `s[l] != s[r]` then `cost[l][r]++`

📌 `cost[l][r] = cost[l+1][r-1] + abhi ka`

📌 to baad ke left pehle process krenge

📌 ab wahi purana dp partition

Implementation

```
class Solution {
public:
    int palindromePartition(string s, int k) {

        //basically to dp partition method hi use krunga isme
        //dp[i][k] waala

        //but isme ek additional cheez he which is
        //to find cost to make palindrome

        //basically cost[l][r]=cost[l+1][r-1]
        //if s[l]!=s[r] then cost[l][r]++

        int n = s.size();
        vector<vector<int>> cost(n, vector<int>(n, 0));
```

```

//cost[l][r]=cost[l+1][r-1]+abhi ka
//to baad ke left pehle process krenge

for(int l = n-1; l >= 0; l--) {
    for(int r = l+1; r < n; r++) {
        cost[l][r] = cost[l+1][r-1];
        if(s[l] != s[r]) cost[l][r]++;
    }
}

//ab wahi purana dp partition

vector<vector<int>> dp(n+1, vector<int>(k+1, INT_MAX));
dp[0][0] = 0;

for(int i = 1; i <= n; i++) {
    for(int parts = 1; parts <= k; parts++) {

        for(int j = parts-1; j < i; j++) {

            int lcost = cost[j][i-1];

            if(dp[j][parts-1] == INT_MAX) continue;
            int curr = dp[j][parts-1] + lcost;

            dp[i][parts] = min(dp[i][parts], curr);
        }
    }
}

return dp[n][k];
};

```